

CS-405: Deep Learning

Week 6 - Sequential Models: From RNNs to GRUs

Dr. Sajjad Hussain

SEECS, NUST

February 24th, 2026

Outline - Part 1

- 1 Introduction to Sequential Models
- 2 Recurrent Neural Networks (RNNs)
- 3 Backpropagation Through Time (BPTT)
- 4 Gated Recurrent Units (GRU)

What Are Sequential Models?

Definition

Models designed to handle data where **order matters** - the sequence of inputs provides important information.

- **Key Characteristic:**

- Inputs are not independent
- Current output depends on previous inputs
- Sequences can have **variable length**

- **Real-World Analogy:**

“Reading a book - you can't understand Chapter 10 without having read Chapters 1-9!”

Why Do We Need Sequential Models?

Traditional ML Assumption

i.i.d. data (independent and identically distributed)

- Each data point independent
- Order doesn't matter

Real World is Sequential!

- Language: Words in a sentence
- Time: Stock prices over days
- Video: Frames in a movie
- Speech: Audio signals
- Biology: DNA sequences

We need models that understand order and context!

Common Applications of Sequential Models

Domain	Application	Example
NLP	Machine Translation	English → French
NLP	Sentiment Analysis	"This movie is great" (Positive)
NLP	Named Entity Recognition	"Apple" → Company or Fruit?
Time Series	Stock Prediction	Future price based on history
Time Series	Weather Forecasting	Next week's temperature
Vision	Video Classification	Activity recognition
Biology	DNA Sequencing	Gene prediction
Speech	Speech Recognition	Audio → Text

Why Standard Neural Networks Fail on Sequential Data

① Fixed Input Size

- Traditional Feed Forward Networks (FNNs) require fixed-size inputs
 - Sentences have variable lengths
- ⇒ Can't handle sequences of different lengths

② No Parameter Sharing

- FNNs learn separate weights for each position
 - Pattern “France → French” should work anywhere in sentence
- ⇒ Can't generalize across positions

③ No Memory

- Each prediction is independent
 - Can't use context from previous words
- ⇒ Treats each word in isolation

Concrete Example - The Failure Case

Sentence A

“I grew up in **France**. I speak fluent

Sentence B

“I visited **France** last summer. The weather was great.”

What a feedforward network sees:

Sentence A: [I, grew, up, in, France, ., I, speak, fluent, ?]

Sentence B: [I, visited, France, last, summer, ., The, weather, was, great]

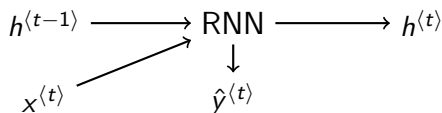
- Different lengths → can't use same architecture
- Position of “France” matters (5th vs 3rd word)
- Cannot learn that “France” implies “French” regardless of position

What we need: A network that processes words one by one and maintains context!

The RNN Solution - Adding Memory

Core Idea

Process sequences step-by-step while maintaining a **hidden state** (memory)



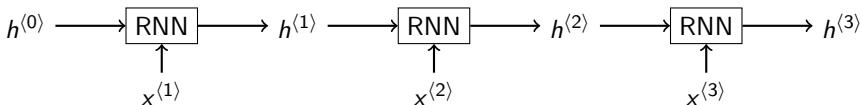
- **At each time step:**

- 1 Read current input ($x^{\langle t \rangle}$)
- 2 Update memory using previous memory ($h^{\langle t-1 \rangle}$)
- 3 Produce output ($\hat{y}^{\langle t \rangle}$) based on current memory

The Hidden State - Network's Memory

Definition

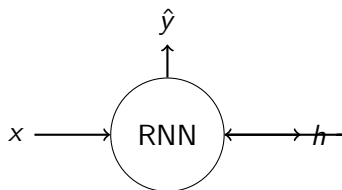
The **hidden state** $h^{(t)}$ is a vector that summarizes everything the network has seen up to time t .



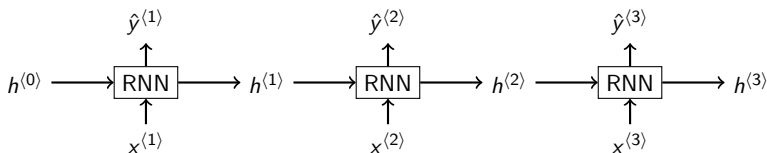
- $h^{(0)}$: Initial memory (usually zeros)
- $h^{(1)}$: Memory after seeing $x^{(1)}$
- $h^{(2)}$: Memory after seeing $x^{(1)}, x^{(2)}$
- $h^{(3)}$: Memory after seeing $x^{(1)}, x^{(2)}, x^{(3)}$

Unfolding the RNN Through Time

Compact Representation (Loop):



Unfolded Through Time:



Key Insight: The SAME RNN cell (with same weights) is used at every time step!

RNN Terminology and Notation

Symbol	Meaning	Role
$x^{(t)}$	Input at time step t	Current word/data point
$h^{(t)}$	Hidden state at time t	Memory after processing up to t
$\hat{y}^{(t)}$	Output at time t	Prediction at this step
$h^{(0)}$	Initial hidden state	Usually zeros (no memory)
T_x	Sequence length	Number of time steps
n_x	Input dimension	Size of input vector
n_h	Hidden dimension	Size of memory vector
n_y	Output dimension	Size of output vector

Example

For word-level model with 10,000 words: $n_x = 10,000$

Hidden state size 100: $n_h = 100$

Binary classification: $n_y = 1$

The RNN Weight Matrices

Three Key Weight Matrices (ALL SHARED across time)

Matrix	Dimensions	Purpose
W_{hx}	$n_h \times n_x$	Maps input to hidden state
W_{hh}	$n_h \times n_h$	Transforms previous hidden state
W_{yh}	$n_y \times n_h$	Maps hidden state to output

Biases

- b_h : Hidden bias ($n_h \times 1$)
- b_y : Output bias ($n_y \times 1$)

Example

With $n_h = 100$, $n_x = 10,000$, $n_y = 1$:

- W_{hx} : $100 \times 10,000$ (1M parameters!)
- W_{hh} : 100×100 (10K parameters)

RNN Forward Pass - The Equations

Step 1: Update hidden state (memory)

$$h^{\langle t \rangle} = \tanh(W_{hx} \cdot x^{\langle t \rangle} + W_{hh} \cdot h^{\langle t-1 \rangle} + b_h)$$

Step 2: Produce output (prediction)

$$\hat{y}^{\langle t \rangle} = \sigma(W_{yh} \cdot h^{\langle t \rangle} + b_y) \quad (\text{for binary classification})$$

Alternative for multi-class

$$\hat{y}^{\langle t \rangle} = \text{softmax}(W_{yh} \cdot h^{\langle t \rangle} + b_y)$$

Understanding the Hidden State Equation

$$h^{(t)} = \tanh(\underbrace{W_{hx}x^{(t)}}_{\text{New info}} + \underbrace{W_{hh}h^{(t-1)}}_{\text{Past memory}} + \underbrace{b_h}_{\text{Bias}})$$

- $W_{hx}x^{(t)}$: Processes current input
- $W_{hh}h^{(t-1)}$: Incorporates past memory
- b_h : Bias term
- $+$: Combines current and past
- $\tanh$: Squashes values to $[-1, 1]$

Interpretation: *The new memory is a **combination** of:*

- *What we see now ($x^{(t)}$)*
- *What we remember from before ($h^{(t-1)}$)*

Concrete Example - Named Entity Recognition

Task

Identify if each word is a person name (1) or not (0)

Sentence: “Martin Luther King gave a speech”

t	Word	True Label ($y^{(t)}$)
1	“Martin”	1
2	“Luther”	1
3	“King”	1
4	“gave”	0
5	“a”	0
6	“speech”	0

Model dimensions:

- Vocabulary size: 10,000 ($n_x = 10,000$)
- Hidden state size: 100 ($n_h = 100$)
- Output: Binary ($n_y = 1$)

Step-by-Step Forward Pass - $t=1$ (“Martin”)

Initial state: $h^0 = [0, 0, \dots, 0]$ (100 zeros)

Step 1: Compute hidden state

$$z^1 = W_{hx} \cdot x^1 + W_{hh} \cdot h^0 + b_h$$
$$h^1 = \tanh(z^1)$$

Hypothetical: h^1 activates neurons for “person name beginning”

Step 2: Compute output

$$\hat{y}^1 = \sigma(W_{yh} \cdot h^1 + b_y) = 0.82$$

True label: $y^1 = 1$ (Martin is a person)

Loss (Binary Cross-Entropy)

$$L^1 = -[1 \cdot \log(0.82) + 0 \cdot \log(0.18)] = 0.20$$

Step-by-Step Forward Pass - $t=2$ (“Luther”)

Now use previous memory h^1 :

Step 1: Update hidden state

$$z^2 = W_{hx} \cdot x^2 + W_{hh} \cdot h^1 + b_h$$
$$h^2 = \tanh(z^2)$$

What's happening: h^2 combines “Luther” with memory of “Martin”

Step 2: Compute output

$$\hat{y}^2 = \sigma(W_{yh} \cdot h^2 + b_y) = 0.79$$

True label: $y^2 = 1$ (Luther is part of person name)

Loss

$$L^2 = -[1 \cdot \log(0.79) + 0 \cdot \log(0.21)] = 0.24$$

Forward Pass Summary

t	Word	$\hat{y}$	y	Loss
1	"Martin"	0.82	1	0.20
2	"Luther"	0.79	1	0.24
3	"King"	0.81	1	0.21
4	"gave"	0.35	0	0.43
5	"a"	0.28	0	0.33
6	"speech"	0.31	0	0.37
Total Loss				1.78

- Loss at each step measures prediction error
- Total loss = sum of all step losses
- We'll use this total loss to update weights

The Combined Weight Matrix Trick

Combined (for efficiency)

$$h^t = \tanh \left(W_h \cdot \begin{bmatrix} h^{t-1} \\ x^t \end{bmatrix} + b_h \right)$$

Where:

- $W_h = [W_{hh} \quad W_{hx}]$ (concatenated horizontally)
- $\begin{bmatrix} h^{t-1} \\ x^t \end{bmatrix}$ (concatenated vertically)

Example

With $n_h = 100$, $n_x = 10,000$:

- W_h : $100 \times (100 + 10,000) = 100 \times 10,100$
- Combined input: $(100 + 10,000) \times 1 = 10,100 \times 1$

Benefit: One matrix multiplication instead of two!

How Do We Train RNNs?

The Challenge

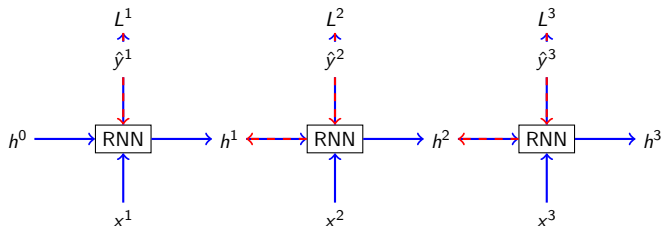
The same weights (W_{hx} , W_{hh} , W_{yh}) are used at **EVERY** time step!

The Solution: Backpropagation Through Time (BPTT)

- 1 **Unfold** the RNN through all time steps
- 2 Compute loss at each step
- 3 Backpropagate through **ALL** steps
- 4 **Sum** gradients from each step
- 5 Update weights once

Key Insight: The gradient at time t depends on ALL previous time steps!

Visualizing BPTT



Total Gradient:

$$\frac{\partial L}{\partial W} = \frac{\partial L^1}{\partial W} + \frac{\partial L^2}{\partial W} + \frac{\partial L^3}{\partial W}$$

The Chain Rule in BPTT

For loss at time $t = 3$:

$$\frac{\partial L^3}{\partial W_{hh}} = \frac{\partial L^3}{\partial \hat{y}^3} \cdot \frac{\partial \hat{y}^3}{\partial h^3} \cdot \frac{\partial h^3}{\partial W_{hh}}$$

But h^3 depends on h^2 , which depends on h^1 :

$$\frac{\partial h^3}{\partial W_{hh}} = \underbrace{\frac{\partial h^3}{\partial W_{hh}} \Big|_{\text{explicit}}}_{\text{direct}} + \underbrace{\frac{\partial h^3}{\partial h^2} \cdot \frac{\partial h^2}{\partial W_{hh}}}_{\text{via } h^2}$$

And $\frac{\partial h^2}{\partial W_{hh}}$ expands further:

$$\frac{\partial h^2}{\partial W_{hh}} = \frac{\partial h^2}{\partial W_{hh}} \Big|_{\text{explicit}} + \frac{\partial h^2}{\partial h^1} \cdot \frac{\partial h^1}{\partial W_{hh}}$$

The Full Expansion - Why We Add Terms

Continuing back to $t = 1$:

$$\begin{aligned}\frac{\partial h^3}{\partial W_{hh}} &= \underbrace{\frac{\partial h^3}{\partial W_{hh}} \Big|_{\text{explicit}}}_{\text{from } t=3} \\ &+ \underbrace{\frac{\partial h^3}{\partial h^2} \cdot \frac{\partial h^2}{\partial W_{hh}} \Big|_{\text{explicit}}}_{\text{from } t=2} \\ &+ \underbrace{\frac{\partial h^3}{\partial h^2} \cdot \frac{\partial h^2}{\partial h^1} \cdot \frac{\partial h^1}{\partial W_{hh}} \Big|_{\text{explicit}}}_{\text{from } t=1}\end{aligned}$$

The Full Expansion - Why We Add Terms

Why addition? Each term represents a different path through which W_{hh} affects h^3 :

- **Path 1:** Direct effect at $t=3$
- **Path 2:** Effect at $t=2$, then propagated to $t=3$
- **Path 3:** Effect at $t=1$, propagated through $t=2$ to $t=3$

We ADD because these are independent pathways - the total effect is the sum of all pathways!

Computing $\frac{\partial h^t}{\partial h^{t-1}}$

Recall: $h^t = \tanh(W_{hx}x^t + W_{hh}h^{t-1} + b_h)$

Let $a^t = W_{hx}x^t + W_{hh}h^{t-1} + b_h$

Then:

$$\frac{\partial h^t}{\partial h^{t-1}} = \frac{\partial h^t}{\partial a^t} \cdot \frac{\partial a^t}{\partial h^{t-1}}$$

- $\frac{\partial h^t}{\partial a^t} = \text{diag}(1 - (h^t)^2)$ (derivative of $\tanh$)
- $\frac{\partial a^t}{\partial h^{t-1}} = W_{hh}^T$ (derivative of linear term)

Therefore:

$$\frac{\partial h^t}{\partial h^{t-1}} = \text{diag}(1 - (h^t)^2) \cdot W_{hh}^T$$

The Vanishing Gradient Problem

For a path from h^1 to h^T :

$$\frac{\partial h^T}{\partial h^1} = \prod_{t=2}^T \left[\text{diag}(1 - (h^t)^2) \cdot W_{hh}^T \right]$$

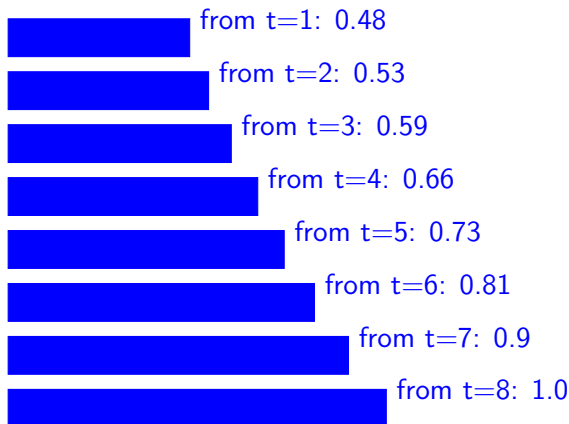
Two factors that shrink gradients:

Factor	Effect
$\text{diag}(1 - (h^t)^2)$	Each entry is between 0 and 1
W_{hh}^T	If eigenvalues < 1 , shrinks further

Result: For long sequences, $\frac{\partial h^T}{\partial h^1} \approx 0$!

Visualizing Vanishing Gradients

Gradient strength from $t=8$ back to earlier steps:



Vanishing Gradients for Longer Sequences

For a 50-step sequence:

from $t=50$: 1.0

...

from $t=1$: 0.005

The network effectively has NO MEMORY of early words!

Concrete Example - The “France” Problem

Sentence

“I grew up in **France**. I moved to Germany for work. After 10 years, I now speak fluent **French**.”

What should happen:

- “France” at position 5 should influence “French” at position 19
- Need gradient to flow back 14 steps

What actually happens in RNN:

- Gradient from $t=19$ to $t=5$ is multiplied ~ 14 times
- Each multiplication shrinks by, say ~ 0.8
- After 14 steps: $(0.8)^{14} \approx 0.044$

Result: The network receives almost no signal that “France” matters for predicting “French”!

The Exploding Gradient Problem

The Other Extreme: When W_{hh} has eigenvalues > 1

$$\frac{\partial h^T}{\partial h^1} \approx \|W_{hh}^T\|^{T-1}$$

If $\|W_{hh}^T\| = 1.1$:

Steps	Growth Factor
10	2.59
20	6.73
50	117.4
100	13,780

Symptoms:

- Gradients become NaN (too large)
- Weights update wildly
- Training diverges

Solution: Gradient clipping (cap gradients at max value)

RNN Limitations Summary

Problem	Cause	Consequence
Vanishing Gradients	Repeated multiplication by values < 1	Can't learn long-range dependencies
Exploding Gradients	Repeated multiplication by values > 1	Training instability, NaN values
Short Memory	Effective context window $\sim 5-10$ steps	Fails on long sentences

This motivates more advanced architectures like GRU and LSTM!

The GRU Solution - Intelligent Memory Management

Core Idea

Give the network **control** over what to remember and what to forget

Gate	Symbol	Purpose
Update Gate	z^t	Decides how much of old memory to keep
Reset Gate	r^t	Decides how much past context matters now

Analogy: A Smart Notebook

- Keep important pages forever (update gate ≈ 1)
- Erase irrelevant pages (reset gate ≈ 0)
- Write new information when needed

GRU Variables

Symbol	Description
$h^{(t-1)}$	Previous hidden state (memory from past)
$x^{(t)}$	Current input (new information)
$h^{(t)}$	New hidden state (updated memory)
$\tilde{h}^{(t)}$	Candidate hidden state (proposed new memory)
$z^{(t)}$	Update gate (0 to 1) - how much to keep vs. update
$r^{(t)}$	Reset gate (0 to 1) - how much past to forget

GRU - The Update Gate (z^t)

$$z^{(t)} = \sigma(W_z \cdot [h^{(t-1)}, x^{(t)}] + b_z)$$

- **Inputs:** Previous memory ($h^{(t-1)}$) and current input ($x^{(t)}$)
- **Activation:** Sigmoid (σ) outputs values between 0 and 1

Interpretation:

Value	Meaning
$z^{(t)} \approx 1$	Keep old memory, ignore new info
$z^{(t)} \approx 0$	Replace old memory with new info

Think of $z^{(t)}$ as a **knob** controlling how much to update.

GRU - The Reset Gate (r^t)

$$r^{\langle t \rangle} = \sigma(W_r \cdot [h^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_r)$$

- **Inputs:** Same as update gate
- **Activation:** Sigmoid (0 to 1)

Interpretation:

Value	Meaning
$r^{\langle t \rangle} \approx 1$	Use all of past memory
$r^{\langle t \rangle} \approx 0$	Ignore past memory, start fresh

Think of $r^{\langle t \rangle}$ as a **filter** that decides how much past context matters right now.

GRU - Candidate Hidden State ($\tilde{h}^t$)

$$\tilde{h}^{(t)} = \tanh(W_h \cdot [r^{(t)} \odot h^{(t-1)}, x^{(t)}] + b_h)$$

Where $\odot$ is element-wise multiplication.

What happens:

- ➊ Reset gate r^t filters the past memory
 - ➋ Filtered memory $r^t \odot h^{t-1}$ combines with current input
 - ➌ $\tanh$ creates a candidate (like RNN's hidden state)
- If $r^t \approx 0$: Past is ignored, candidate based only on current input
 - If $r^t \approx 1$: Full past memory used

GRU - Final Hidden State (h^t)

The Magic Formula:

$$h^{\langle t \rangle} = (1 - z^{\langle t \rangle}) \odot \tilde{h}^{\langle t \rangle} + z^{\langle t \rangle} \odot h^{\langle t-1 \rangle}$$

This is a weighted average between:

- **New candidate** $\tilde{h}^t$ (weighted by $1 - z^t$)
- **Old memory** h^{t-1} (weighted by z^t)

Interpretation:

z^t value	Meaning
0.9	Keep 90% of old, add 10% new
0.1	Keep 10% of old, add 90% new
0.5	Equal mix of old and new

GRU Example - Long-Range Dependency

Sentence

“I grew up in **France**. I moved to Germany for work. After 10 years, I now speak fluent **French**.”

At $t=5$ (seeing “France”):

- Update gate $z^5 \approx 0.95$ for country-information dimension
- This dimension now strongly encodes “France”

At $t=6-18$ (filler words):

- For the country dimension, $z^t \approx 0.95$ at each step
- Memory decays slowly: $0.95^{14} \approx 0.49$ (still strong!)

At $t=19$ (seeing “French”):

- Reset gate $r^{19} \approx 0.9$ (past is relevant)
- Candidate combines “French” + preserved “France” memory
- Network recognizes: “France” + “French” = language!

RNN vs GRU - Numerical Comparison

RNN Memory Decay

Start: 1.0

$$t = 6 : \times 0.8 = 0.8$$

$$t = 7 : \times 0.8 = 0.64$$

$$t = 8 : \times 0.8 = 0.51$$

$\vdots$

$$t = 19 : 1.0 \times (0.8)^{14} = \mathbf{0.044}$$

GONE!

GRU Memory Decay ($z=0.95$)

Start: 1.0

$$t = 6 : \times 0.95 = 0.95$$

$$t = 7 : \times 0.95 = 0.90$$

$$t = 8 : \times 0.95 = 0.86$$

$\vdots$

$$t = 19 : 1.0 \times (0.95)^{14} = \mathbf{0.49}$$

STRONG!

The difference: GRU can CHOOSE to preserve information by setting z close to 1!

How GRU Solves Vanishing Gradients

In RNN:

$$\frac{\partial h^t}{\partial h^{t-1}} = \text{diag}(1 - (h^t)^2) \cdot W_{hh}^T$$

In GRU:

$$\frac{\partial h^t}{\partial h^{t-1}} = \text{diag}(z^t) + \text{terms from reset gate and candidate}$$

Aspect	RNN	GRU
Gradient path	Single multiplicative	Additive + multiplicative
Control	None	Can set $z^t \approx 1$
Dimension mixing	Full matrix	Element-wise for direct path

Understanding $\text{diag}(z^t)$

What is z^t ?

$$z^t = [z_1, z_2, \dots, z_{n_h}]^T$$

What is $\text{diag}(z^t)$?

$$\text{diag}(z^t) = \begin{bmatrix} z_1 & 0 & \dots & 0 \\ 0 & z_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & z_{n_h} \end{bmatrix}$$

Why matrix form?

- Need Jacobian (matrix of partial derivatives)
- Shows how each dimension of h^t depends on each dimension of h^{t-1}
- Diagonal means each dimension evolves independently in the direct path

Gradient Flow in GRU

For a single dimension with $z^t \approx 1$:

$$\frac{\partial h_i^T}{\partial h_i^1} \approx \prod_{t=2}^T z_i^t \approx 1^{T-1} = 1$$

The gradient for that dimension survives!

For dimensions with $z^t \approx 0$:

$$\frac{\partial h_i^T}{\partial h_i^1} \approx \prod_{t=2}^T z_i^t \approx 0$$

These dimensions update normally and forget the past.

Key Insight: Different dimensions can have different “forgetting rates” - the network learns which information to preserve!

GRU Summary - Why It's Better

Feature	RNN	GRU
Memory control	None	Gates control what to keep/forget
Gradient path	Multiplicative only	Additive + multiplicative
Long-range dependencies	Fails after ~ 10 steps	Can remember hundreds
Vanishing gradients	Inevitable	Preventable with $z \approx 1$
Parameters	$3n_h^2 + \dots$	Slightly more (gates)
Training	Difficult on long sequences	Much more stable

The Bottom Line: GRU gives the network the **tools to decide what's important** and preserve it, rather than blindly overwriting everything at each step.

When to Use RNN vs GRU

Use Simple RNN when:

- Sequences are short (< 10 steps)
- Long-range dependencies aren't critical
- Computational resources are very limited
- You want a baseline model

Use GRU when:

- Sequences are long
- Long-range dependencies matter
- You need stable training
- You have enough data to learn gate parameters

Key Takeaways

- **Sequential Models:** Needed when order matters; FNNs fail due to fixed size and no memory
- **RNNs:** Process sequences step-by-step with hidden state; share weights across time
- **BPTT:** Unfold through time, sum gradients, update shared weights
- **Vanishing Gradients:** Repeated multiplication causes gradients to shrink; RNNs forget early inputs
- **GRU:** Adds gates to control memory; z^t gate preserves important information
- **Why GRU Works:** Additive gradient path via $\text{diag}(z^t)$ preserves gradients for important dimensions

References and Further Reading



Cho et al. (2014)

“Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”

Original GRU paper



Hochreiter & Schmidhuber (1997)

“Long Short-Term Memory”

Original LSTM paper



Goodfellow, Bengio, Courville (2016)

“Deep Learning” Book, Chapter 10



Colah's Blog

“Understanding LSTM Networks”

Excellent visual explanation



Stanford CS224n Lecture Notes

“Natural Language Processing with Deep Learning”